

Project Name	Accessible MFA — Multi-Factor Authentication for Visually Impaired Users
Module	CS3053 — Computer Security — Group Project (Semester 5)
Assignment	Submission 1 — Individual Design Work
Group Name	CRYPTIX
Student ID	230135A
Student Name (with initials)	DEWDUNIKA D.R.K.W.M.V.V
Group Slice Owned	Member 1 — Accounts & Factor 1 (knowledge factor: password/PIN)
Date	15 September 2026

Individual Design Contribution

Accounts & Factor 1 — User Registration, Login, Sessions and Knowledge-Factor Authentication

Contents

1. Scope of my contribution
2. Assumptions
3. Requirements
4. System context (design artifact 1)
5. Data design — entity-relationship model (design artifact 2)
6. Behavioural design — sequence diagrams (design artifacts 3 & 4)
7. Behavioural design — session/auth state machine (design artifact 5)
8. API design
9. Design decisions and justification
10. Accessibility rationale specific to this slice
11. Security evaluation against the threat model
12. Known limitations and interaction with other slices
13. References

1. Scope of my contribution

I own the **Accounts & Factor 1** slice end to end: design, implementation, tests and this report. It covers:

Area	What I designed / will implement
Data	The <code>User</code> record (credential storage) and the <code>Session</code> record that other slices (Factor 2, Factor 3, orchestration) read and extend.
Endpoints	<code>POST /register</code> , <code>POST /login</code> , <code>POST /logout</code> , <code>GET /me</code> .

Factor 1 logic	Knowledge-factor authentication: password or PIN, chosen by the user at registration so that a numeric-keypad-only entry method is always available.
Security controls owned here	Argon2id password/PIN hashing, credential validation policy, and the account-side half of failed-attempt tracking that Member 5's rate-limit/lockout middleware consumes.

Everything else (TOTP/email OTP, WebAuthn, client accessibility, orchestration/lockout/audit) belongs to Members 2–5 and appears here only as necessary context.

2. Assumptions

None of the assumptions below removes a required security or accessibility property.

A1 — User population. Covers both blind users (audio-only) and low-vision users (high contrast/large text). The knowledge factor is *audio-first*, not *audio-only*.

A2 — Assistive technology. Standard keyboard + screen reader (NVDA, JAWS, VoiceOver or TalkBack) — the four readers used by 1,539 respondents in the WebAIM Screen Reader Survey #10 [6]. No unusual assistive hardware assumed.

A3 — Delivery platform. Browser-based web app, not a native mobile app, so browser-native accessibility APIs and TLS can be relied on.

A4 — TLS is provided by deployment, not my code. HTTPS termination is assumed to sit in the deployment environment (e.g. a reverse proxy); my design marks cookies `Secure` but does not implement TLS itself.

A5 — Single relational database. One database (e.g. PostgreSQL/SQLite), no horizontal scaling — appropriate for an academic prototype, not a production service.

A6 — Recovery email is not an MFA factor. It exists only for account recovery (Member 5), because NIST 800-63B says email **SHALL NOT** be used for out-of-band authentication [1, Section 3.1.3.1]. Flagged further in Section 12, since Member 2's slice lists email OTP as an option.

A7 — Threat model boundary. Local device/OS compromise and malicious AT extensions are out of scope, per the group threat model. This slice defends remote credential-guessing and database-leak scenarios (T1, T8, T9, T12).

3. Requirements

3.1 Functional requirements (FR)

ID	Requirement
FR1	A new user can register with a username and <i>either</i> a password <i>or</i> a numeric PIN.
FR2	A registered user can authenticate Factor 1 with the same credential type they registered with.
FR3	On successful Factor 1 verification, the system starts a session and reports which factor is required next.

FR4	A user can log out, invalidating their session server-side.
FR5	An authenticated user can retrieve their own account summary (/me), including which factors they have enrolled.

3.2 Accessibility requirements (AR)

ID	Requirement	Standard
AR1	No step in Factor 1 may require solving a puzzle, transcribing a CAPTCHA, or otherwise recalling/manipulating information beyond the credential itself, unless an alternative or assistive mechanism is offered.	WCAG 2.2 SC 3.3.8 [3]
AR2	Password/PIN fields must accept paste and must not block password-manager autofill.	NIST 800-63B Section 3.1.1.2 [1]; WCAG 2.2 Understanding 3.3.8 [4]
AR3	Every error response's message must be a complete, standalone sentence that makes sense read aloud with no visual context (no reliance on colour or icon alone).	WCAG 2.2 SC 1.4.1, SC 3.3.1 [3]
AR4	No fixed short time limit on completing registration/login; if any timeout exists it must be extendable.	WCAG 2.2 SC 2.2.1 [3]

3.3 Security requirements (SR)

ID	Requirement	Standard
SR1	Passwords used alongside a second factor must be at least 8 characters; a password used alone must never be accepted as sufficient for login (MFA is mandatory).	NIST 800-63B Section 3.1.1.2 [1]
SR2	No composition rules (mixed case/digit/symbol) and no periodic forced expiry.	NIST 800-63B Section 3.1.1.2 [1]
SR3	Credentials are checked against a breached/common-credential blocklist at registration.	NIST 800-63B Section 3.1.1.2 [1]
SR4	Credentials are stored using Argon2id, never reversible encryption or a fast general-purpose hash.	OWASP Password Storage Cheat Sheet [2]
SR5	Failed Factor-1 attempts on one account are capped, feeding Member 5's lockout middleware.	NIST 800-63B Section 3.2.2 [1]
SR6	Session identifiers have ≥ 64 bits of entropy and are rotated on privilege change (each factor completed).	OWASP Session Management Cheat Sheet [7]

4. System context (design artifact 1)

Figure 1 shows where my slice (highlighted) sits in the system, including the trust boundaries from the group threat model.

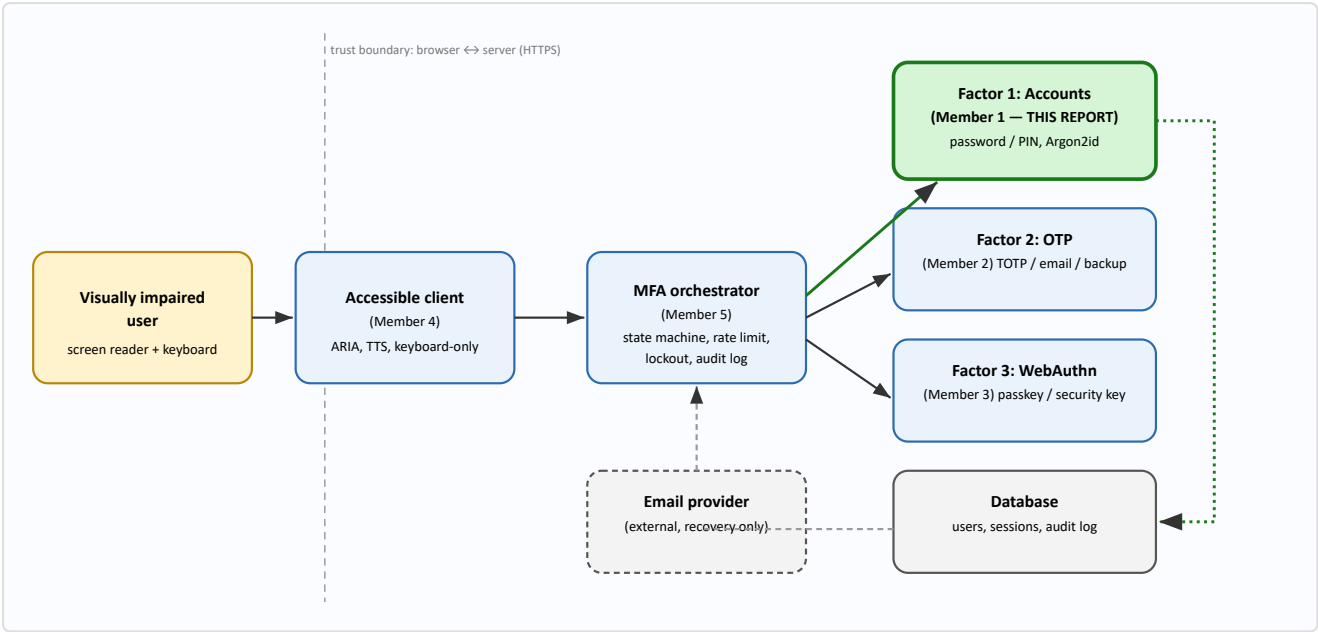


Figure 1. System context diagram (green = component covered by this report). Dashed line marks the browser↔server trust boundary from the group threat model. The dashed arrow from Factor 1 to Database represents credential read/write; the solid line from client to orchestrator represents every screen making its request through the orchestration layer's `requireFactor(n)` middleware.

5. Data design — entity-relationship model (design artifact 2)

Figure 2 shows the two entities I own (`USER` , `SESSION`), crow's-foot notation, plus Member 5's `AUDIT_LOG` for context.

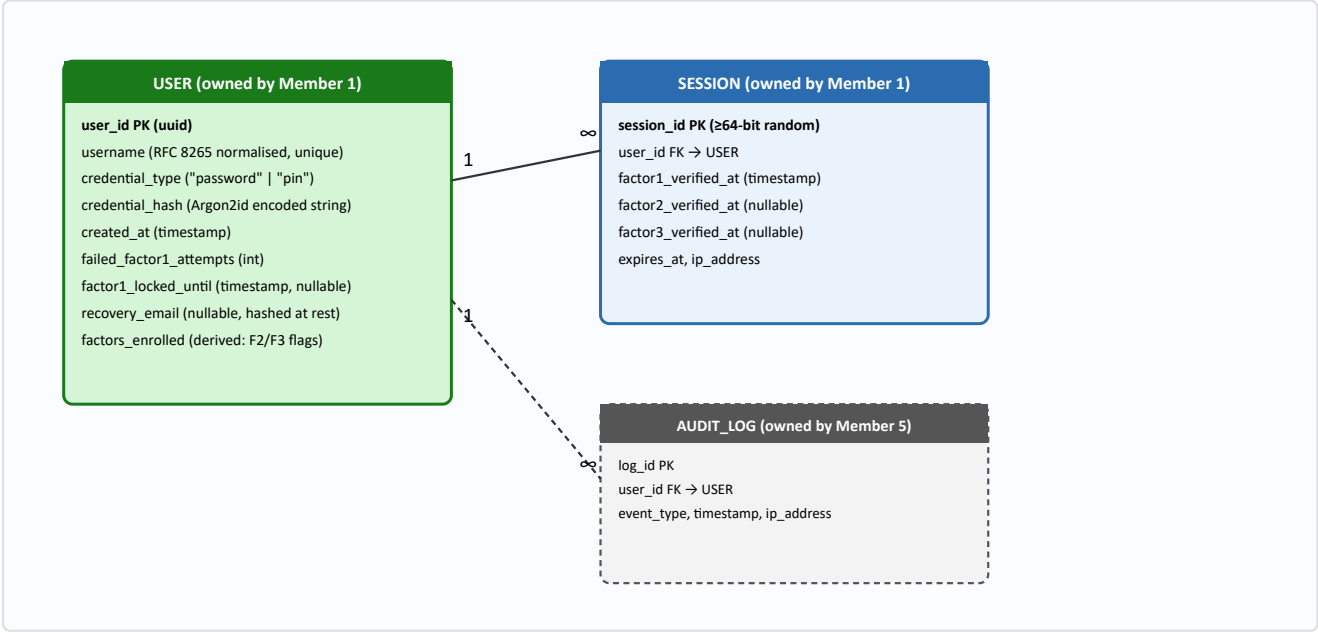


Figure 2. Entity-relationship diagram, crow's-foot cardinality (1 : ∞). Solid entities/edges are owned by this slice; the dashed `AUDIT_LOG` box is Member 5's, shown only to make the `user_id` foreign-key relationship explicit. `credential_hash` stores *either* the password or PIN hash — never both — because a user chooses one Factor-1 method at registration (design decision D5, Section 9).

6. Behavioural design — sequence diagrams (design artifacts 3 & 4)

6.1 Registration

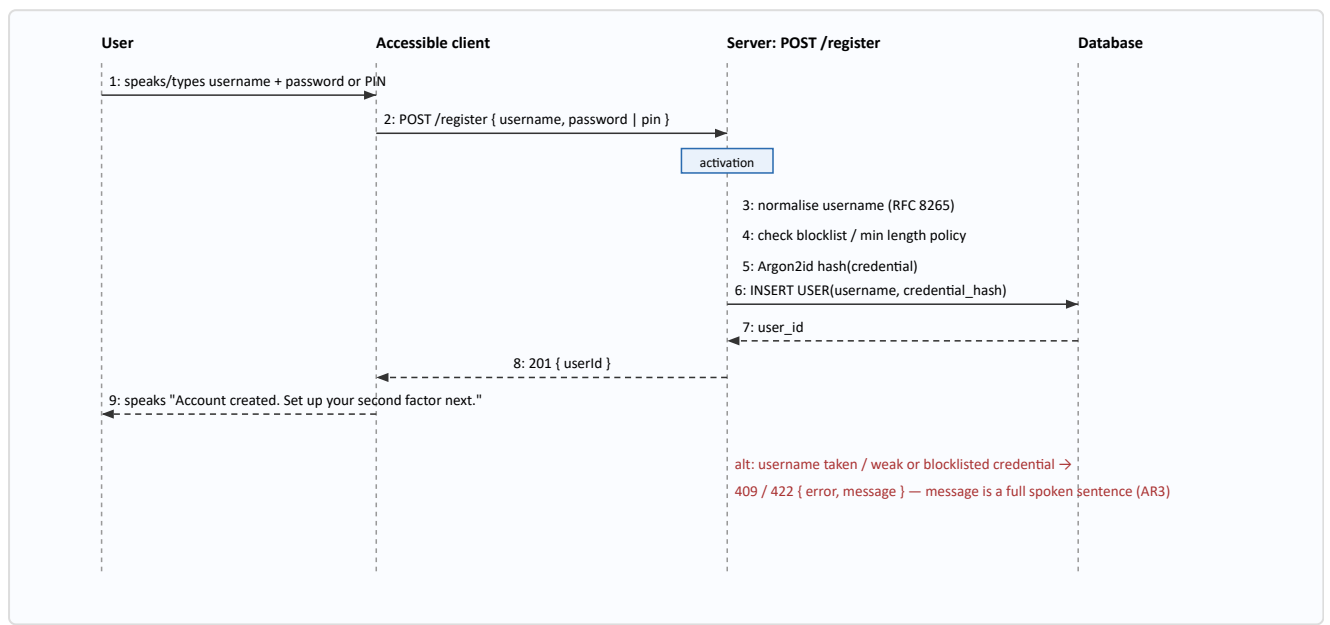


Figure 3. UML sequence diagram — registration (happy path + error alternative). Step 3–5 are the parts I implement inside the `/register` handler.

6.2 Login (Factor 1) and hand-off to Factor 2/3

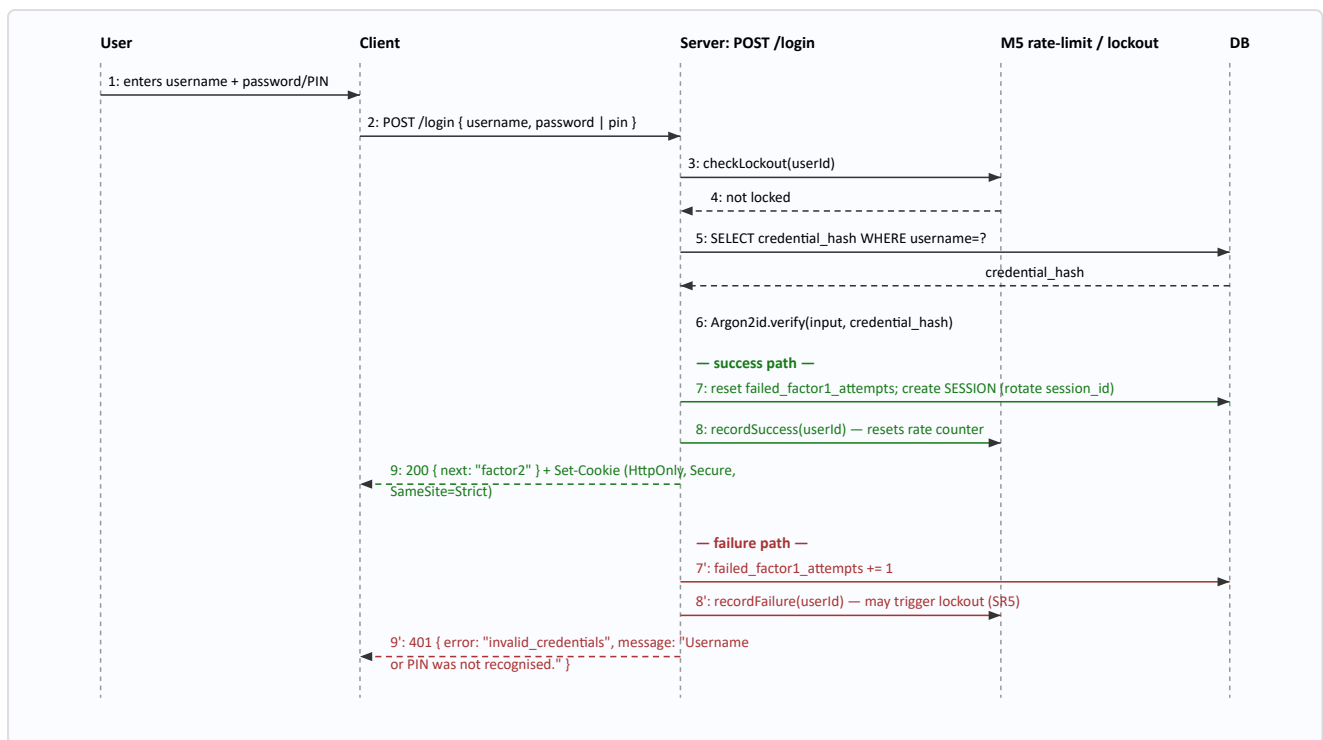


Figure 4. UML sequence diagram — Factor 1 login, showing both outcomes and the exact hand-off point (step 8/9) into Member 5's shared middleware and state machine. Note the error message at 9' never reveals which of username/credential was wrong (mitigates account enumeration, contributes to T1 mitigation).

7. Behavioural design — session/auth state machine (design artifact 5)

Figure 5 shows the part of the shared state machine my slice drives; the full multi-factor machine is owned by Member 5.

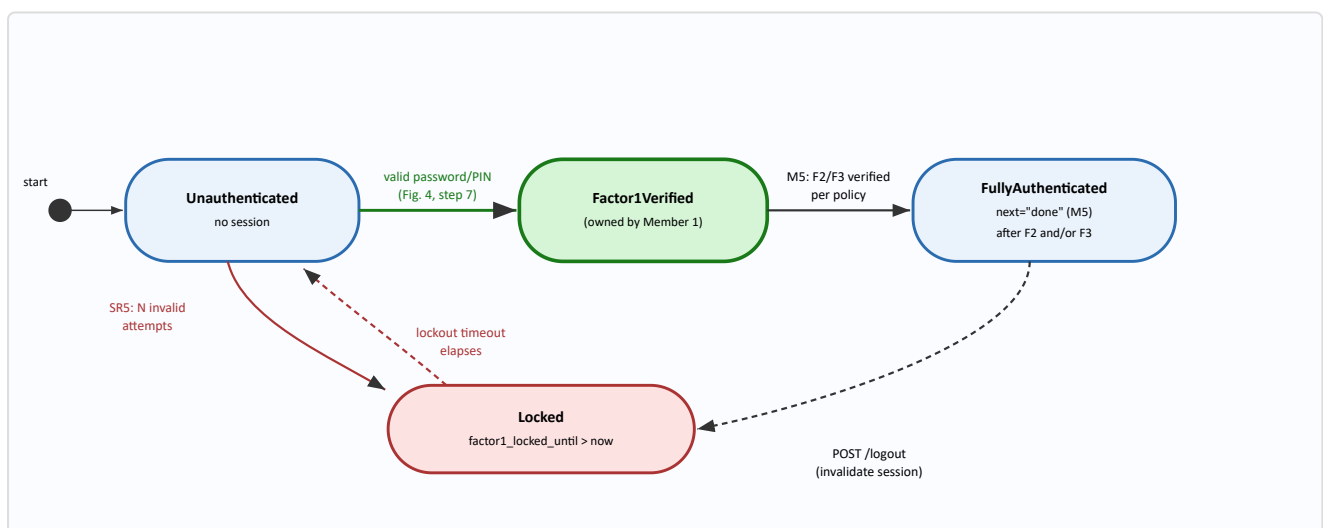


Figure 5. UML state diagram (excerpt). `Unauthenticated` → `Factor1Verified` and `Locked` transitions are implemented by my `/login` handler; `Factor1Verified` → `FullyAuthenticated` is driven by Member 5's `requireFactor(n)` once Factor 2/3 succeed, shown for context only.

8. API design

These four endpoints are the contract I implement, agreed in the group's shared `docs/api-contract.md` before coding began (a deliberate process decision so Member 4 could build the client against a stable contract while my backend was still in progress).

Method	Path	Request	Response	Notes
POST	/register	{ username, password pin }	201 { userId }	Argon2id hash; policy enforced per Section 9 D1–D4
POST	/login	{ username, password pin }	200 { next }	Calls M5 rate limiter (Fig. 4)
POST	/logout	–	204	Server-side session invalidation
GET	/me	–	200 { userId, username, factorsEnrolled }	Requires full auth (all required factors)

Error shape is shared across the whole group API: `{ "error": "<machine_code>", "message": "<spoken sentence>" }`, satisfying AR3.

9. Design decisions and justification

D1 — Argon2id, not bcrypt/PBKDF2/scrypt. Winner of the Password Hashing Competition; OWASP recommends it for its "balanced approach to resisting both side-channel and GPU-based attacks" [2]. Baseline parameters follow OWASP's documented minimum (m=19 MiB, t=2, p=1), tunable to hardware, per NIST's requirement to keep cost "as high as practical" [1, Section 3.1.1.2].

D2 — 8-character minimum, not 15, because MFA is mandatory. NIST 800-63B requires 15 characters only for single-factor use, but permits 8 "for passwords... used as part of multi-factor authentication" [1]. Since Factor 1 alone never grants login (SR1), the shorter, more screen-reader-practical floor applies.

D3 — No composition rules or forced expiry; a compromised-credential blocklist instead. NIST 800-63B forbids both [1, Section 3.1.1.2]. This also serves accessibility: composition rules force error-prone transcription, exactly the "cognitive function test" WCAG SC 3.3.8 asks us to avoid [3, 4].

D4 — Paste and password-manager autofill are allowed on both fields. Required by NIST 800-63B [1]. A 2025 interview study of 33 blind/low-vision users found all of them relied on password managers, and that accessibility gaps in generator/autofill flows pushed some toward reused or predictable passwords instead [9] — blocking paste would make credentials weaker, not stronger.

D5 — A numeric PIN is offered as an alternative to a password (not a replacement), on a standard 12-key layout. ITU-T E.161 standardises a tactile raised dot on the "5" key "to make it easier to use the

keypad... by the visually impaired" [5]; reusing that layout lets tactile/muscle memory transfer. This satisfies WCAG 3.3.8's "Alternative" exception, not just its "Mechanism" exception [4].

D6 — PIN minimum is 6 digits plus a blocklist, not a bare 4-digit default. The largest published PIN study (n=1,220) found 6-digit PINs give no security benefit over 4-digit ones against a throttled attacker (10–100 guesses, matching SR5) *unless* paired with a blocklist covering roughly "10% of the PIN space" [8]. I require both digit count and a common-PIN blocklist, not digits alone.

D7 — Usernames are normalised with RFC 8265 (PRECIS UsernameCaseMapped). NFC normalisation and case-folding keep a username recognisable across different Unicode representations (a real risk from speech-to-text or autocorrect input) and block look-alike-character spoofing [10].

D8 — Failed-attempt counting is reported to Member 5's shared logout middleware, not enforced locally. NIST 800-63B caps consecutive failures at 100 [1, Section 3.2.2]; the group threat model (T1, T10) requires logout keyed on user *and* IP so it can't be weaponised against a victim. One shared policy avoids two competing logout implementations.

D9 — Session cookie is HttpOnly, Secure, SameSite=Strict; ≥64-bit-entropy ID rotated on every factor transition. Follows the OWASP Session Management Cheat Sheet directly, which also warns that `localStorage` / `sessionStorage` tokens are lost to a single XSS bug [7]. Rotating the ID once Factor 1 succeeds also mitigates session fixation (T12).

D10 — Generic, non-enumerating error messages that are still full spoken sentences. The failure response never reveals whether the username or credential was wrong (mitigates account enumeration), while remaining a complete sentence rather than a code or icon, per WCAG's requirement that errors be identified "in text" [3].

10. Accessibility rationale specific to this slice

The WHO estimates at least **2.2 billion people** globally have a near or distance vision impairment [11] — not a small edge case. A contextual inquiry of visually impaired users performing real authentication tasks found login forms measurably slower and more error-prone with a screen reader, largely due to unlabelled fields, unpredictable focus order and inaccessible CAPTCHAs [12] — exactly what WCAG SC 1.3.1, 2.4.3 and 3.3.8 exist to prevent. My Factor-1 forms therefore use: explicit label/ARIA associations (SC 1.3.1), a focus order matching reading order (SC 2.4.3), no CAPTCHA anywhere (rated the single most problematic barrier by screen-reader users in the WebAIM survey [6]), and no short fixed timeout (SC 2.2.1). Manual testing targets JAWS/NVDA on Chrome/Edge first, matching that survey's most common setup [6], then spot-checks VoiceOver.

11. Security evaluation against the threat model

The following rows from the group's shared STRIDE threat model (docs/threat-model.md) are wholly or partly mitigated by this slice:

ID	Threat	My mitigation
T1	Credential stuffing / password guessing	Argon2id (D1) makes offline guessing expensive; failed-attempt reporting feeds shared lockout (D8, SR5); non-enumerating errors (D10) stop username discovery.
T8	Database leak exposes credentials	Only the Argon2id hash is stored, never the plaintext password/PIN (D1); PIN blocklist (D6) reduces value of a leaked hash even after offline cracking.
T9	Session token theft via XSS	HttpOnly cookie (D9) — the token is never reachable from JavaScript even if an XSS bug exists elsewhere in the app.
T12	Session fixation	Session ID is freshly generated and rotated the moment Factor 1 succeeds (D9, Fig. 4 step 7), so a pre-authentication session ID an attacker planted cannot become an authenticated one.

Column "Tested?" for these rows will be completed in Phase 3 with unit tests for D1–D10 and a manual lockout/enumeration test, per the group's testing plan.

12. Known limitations and interaction with other slices

- **Email OTP (Member 2's slice) conflicts with NIST guidance** that email "SHALL NOT be used for out-of-band authentication" [1, Section 3.1.3.1]. My design keeps email out of Factor 1 (A6) and recommends the group treat email OTP as a fallback only, with TOTP/WebAuthn as the default second factor.
- **Factor 1 alone is not phishing-resistant** — NIST 800-63B excludes manually entered authenticator outputs from that category [1, Section 3.2.5]. Phishing resistance in this system comes from Member 3's WebAuthn factor, which must remain genuinely enrollable.

13. References

1. [1] National Institute of Standards and Technology (2025) *SP 800-63B: Digital Identity Guidelines — Authentication and Authenticator Management*, 26 August 2025. Available at: <https://pages.nist.gov/800-63-4/sp800-63b.html> (Accessed: 15 September 2026).
2. [2] OWASP Foundation (n.d.) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 15 September 2026).
3. [3] World Wide Web Consortium (2024) *Web Content Accessibility Guidelines (WCAG) 2.2*, W3C Recommendation, 12 December 2024. Available at: <https://www.w3.org/TR/WCAG22/> (Accessed: 15 September 2026).
4. [4] World Wide Web Consortium (n.d.) *Understanding SC 3.3.8: Accessible Authentication (Minimum)*. Available at: <https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum.html> (Accessed: 15 September 2026).
5. [5] International Telecommunication Union (2001) *ITU-T Recommendation E.161: Arrangement of digits, letters and symbols on telephones and other devices that can be used for gaining access to a telephone network*, 02/2001. Summarised via: Wikipedia contributors, "E.161," Wikipedia, The Free Encyclopedia. Available at: <https://en.wikipedia.org/wiki/E.161> (Accessed: 15 September 2026).
6. [6] WebAIM (2024) *Screen Reader User Survey #10 Results*, December 2023 – January 2024, n=1,539. Available at: <https://webaim.org/projects/screenreadersurvey10/> (Accessed: 15 September 2026).
7. [7] OWASP Foundation (n.d.) *Session Management Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html (Accessed: 15 September 2026).
8. [8] Markert, P., Bailey, D.V., Golla, M., Dürmuth, M. and Aviv, A.J. (2020) 'This PIN Can Be Easily Guessed: Analyzing the Security of Smartphone Unlock PINs', *2020 IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, 18–21 May, pp. 286–303. Preprint: <https://arxiv.org/abs/2003.04868> (Accessed: 15 September 2026).
9. [9] Ponticello, A., Sharevski, F., Anell, S. and Krombholz, K. (2025) *How Blind and Low-Vision Users Manage Their Passwords*, arXiv:2510.13538, 15 October 2025. Available at: <https://arxiv.org/abs/2510.13538> (Accessed: 15 September 2026).
10. [10] Saint-Andre, P. and Blanchet, M. (2017) *RFC 8265: Preparation, Enforcement, and Comparison of Internationalized Strings Representing Usernames and Passwords*, IETF, October 2017. Available at: <https://www.rfc-editor.org/rfc/rfc8265.html> (Accessed: 15 September 2026).
11. [11] World Health Organization (2026) *Blindness and Vision Impairment*, fact sheet, 10 February 2026. Available at: <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment> (Accessed: 15 September 2026).
12. [12] Dosono, B., Hayes, J. and Wang, Y. (2015) '"I'm Stuck!": A Contextual Inquiry of People with Visual Impairments in Authentication', *Proceedings of the Eleventh Symposium on Usable Privacy and Security (SOUPS 2015)*, Ottawa, Canada, 22–24 July, pp. 151–168. Available at: <https://www.usenix.org/conference/soups2015/proceedings/presentation/dosono> (Accessed: 15 September 2026).